# ISLS Extension Phase 2 v1.0.0

C16: Topological-Spectral Orbit Core
C17: Structured Persistence Layer

Absorbing `kernel-k13` and `merkaba-store`
into the Intelligent Semantic Ledger Substrate

Sebastian Klemm

16 March 2026

**Abstract**

This document specifies two additional extension crates for ISLS: (1) `isls-topology` (C16), absorbing the topological-spectral machinery from `kernel-k13` of the MERKABA Factory workspace, providing spectral graph analysis via the graph Laplacian, signal propagation via continuous-time quantum walks (CTQW), and phase synchronization via Kuramoto coupling; and (2) `isls-store` (C17), absorbing the structured SQLite persistence model from `merkaba-store`, upgrading ISLS from filesystem-based storage to a relational persistence layer with projects, runs, crystals, settings, and maintenance operations.

C16 constitutes the *orbit core*: it transforms the persistent graph from a static data structure into a dynamic spectral-topological system whose invariants feed directly into constraint extraction, gate evaluation, and crystal topological signatures. C17 constitutes the *product shell*: it makes ISLS operable as a persistent workspace rather than a stateless pipeline.

All extensions preserve invariants I1–I20 and all existing tests.

**Status:** Implementation Specification.

**Parent:** ISLS v1.0.0 + Extension Architecture v1.0.0.

**Absorbs:** `kernel-k13`, `merkaba-store`.

# Contents


**I Foundations** 4

**1 Scope and Absorption Policy** 4
- 1.1 What Is Absorbed . . . 4
- 1.2 What Is Not Absorbed . . . 4
- 1.3 Absorption Criterion . . . 4

**2 Invariant Preservation** 4

**II Crate C16: `isls-topology`** 4

**3 Purpose** 5

**4 Mathematical Foundations** 5
- 4.1 Graph Laplacian . . . 5
- 4.2 Continuous-Time Quantum Walk (CTQW) . . . 6
- 4.3 Kuramoto Synchronization . . . 7
- 4.4 Discrete Topology Logic (DTL) . . . 7

**5 Integration with Existing Layers** 8
- 5.1 Feeding Constraint Extraction (L2) . . . 8
- 5.2 Feeding Gate Evaluation (L3) . . . 8
- 5.3 Feeding the Spiral Scheduler (C15) . . . 8
- 5.4 Feeding Crystal Signatures . . . 8

**6 Fixpoint Detection** 9

**7 Deduplication** 9

**8 Computational Complexity** 9

**9 Rust API** 10

**10 Acceptance Tests** 11

**III Crate C17: `isls-store`** 12

**11 Purpose** 12

**12 Data Model** 12

**13 Schema** 13

**14 Invariant Preservation** 15

**15 Rust API** 15

**16 CLI Integration** 16

**17 Acceptance Tests** **17**

**IV Dependency Graph and Build Order** **17**

**18 Updated Dependency Graph** **17**

**19 Build Order** **18**

**20 External Dependencies** **18**

**V Implementation Handoff** **18**

**21 Test Count** **19**

**22 Verification Sequence** **19**

**23 Completion Criteria** **19**

# Part I
# Foundations

## 1 Scope and Absorption Policy

### 1.1 What Is Absorbed

From `kernel-k13`:

- Spectral graph analysis (Laplacian, eigenvalues, spectral gap).
- Continuous-time quantum walk (CTQW) signal propagation.
- Kuramoto phase synchronization dynamics.
- Discrete topology logic (DTL) predicates.

From `merkaba-store`:

- SQLite-backed persistence with typed entities.
- Project, run, crystal, settings data model.
- Migration framework.
- Maintenance operations (vacuum, integrity check).

### 1.2 What Is Not Absorbed

- `kernel-qmh` (quantum-hybrid proxy): too speculative for the domain-agnostic core. Remains available as an optional future extension.
- `domain-translator`: language-specific frontends are out of scope for ISLS core.
- `domain-artifact`: DAE synthesis is a domain application, not substrate infrastructure.
- `merkaba-app`: GUI is an application layer, not a substrate component. May be built later on top of C17.

### 1.3 Absorption Criterion

A component from MERKABA Factory is absorbed if and only if:

(i) it is domain-agnostic (no language-specific or UI-specific logic),
(ii) it strengthens at least one existing ISLS invariant or metric,
(iii) it can be expressed as a pure function of the persistent graph and configuration (no external state dependencies),
(iv) it preserves determinism under fixed $(\Sigma, \mathrm{RD})$.

## 2 Invariant Preservation

**Requirement 2.1** (Phase 2 Non-Regression)**.** All extensions in this document **MUST** preserve invariants I1–I20, all acceptance tests AT-01 through AT-20, all extension tests AT-R1 through AT-S5, and all 161+ existing tests. The final test count after Phase 2 **MUST** be $\geq 181$.

# Part II
# Crate C16: `isls-topology`

## 3 Purpose

The topology crate transforms the persistent graph from a static adjacency structure into a dynamic spectral-topological system. It computes three families of invariants that feed into constraint extraction, gate evaluation, crystal signatures, and the spiral scheduler:

(a) **Spectral invariants**: eigenvalues and eigenvectors of the graph Laplacian, spectral gap, algebraic connectivity, Cheeger constant estimate.

(b) **Propagation invariants**: signal spreading speed and reachability via continuous-time quantum walk (CTQW) amplitudes.

(c) **Synchronization invariants**: Kuramoto order parameter measuring phase coherence of entity clusters.

Together these form the *orbit core*: a continuously updated spectral portrait of the graph that serves as the dynamical backbone for all higher layers.

## 4 Mathematical Foundations

### 4.1 Graph Laplacian

**Definition 4.1** (Weighted Adjacency Matrix)**.** Given the persistent graph $G = (V, E, \Sigma_E, \Phi, D)$ with $|V| = n$ vertices, define the weighted adjacency matrix $W \in \mathbb{R}^{n\times n}$ by

$$W_{ij} = \begin{cases} w_e & \text{if edge } e = (v_i, v_j) \in E, \\ 0 & \text{otherwise,} \end{cases}$$

where $w_e$ is the resonance weight from the edge annotation $\Sigma_E(e)$. For undirected analysis, symmetrize: $\bar{W} = (W + W^\top)/2$.

**Definition 4.2** (Degree Matrix)**.** The degree matrix $D \in \mathbb{R}^{n\times n}$ is diagonal with

$$D_{ii} = \sum_{j=1}^{n} \bar{W}_{ij}.$$

**Definition 4.3** (Graph Laplacian)**.** The unnormalized graph Laplacian is

$$L = D - \bar{W}.$$

The normalized graph Laplacian is

$$L_{\text{norm}} = I - D^{-1/2}\bar{W}\, D^{-1/2},$$

with convention $D_{ii}^{-1/2} = 0$ when $D_{ii} = 0$.

**Definition 4.4** (Spectral Decomposition)**.** Let $0 = \lambda_0 \le \lambda_1 \le \cdots \le \lambda_{n-1}$ be the eigenvalues of $L$ with corresponding orthonormal eigenvectors $u_0, u_1, \ldots, u_{n-1}$. The *spectral gap* is

$$\Delta = \lambda_1,$$

which measures algebraic connectivity. The *Fiedler vector* is $u_1$, the eigenvector corresponding to $\lambda_1$.

**Proposition 4.1** (Spectral Gap and Connectivity)**.** $\Delta > 0$ *if and only if* $G$ *is connected. A larger* $\Delta$ *indicates stronger connectivity and faster mixing.*

*Proof.* Standard result from spectral graph theory. $\lambda_0 = 0$ with eigenvector $\mathbf{1}$ always. The multiplicity of eigenvalue 0 equals the number of connected components. □

**Definition 4.5** (Cheeger Constant Estimate)**.** The Cheeger constant $h(G)$ is bounded by the spectral gap via the discrete Cheeger inequality:

$$\frac{\Delta}{2} \leq h(G) \leq \sqrt{2\Delta}.$$

We use $\hat{h} = \sqrt{2\Delta}$ as the upper-bound estimate (computable without solving the NP-hard exact Cheeger problem).

## 4.2 Continuous-Time Quantum Walk (CTQW)

**Definition 4.6** (CTQW Propagator)**.** The CTQW propagator on the graph Laplacian is the unitary (for Hermitian $L$) matrix exponential

$$U(t) = e^{-iLt}.$$

For a signal initialized at vertex $j$ (basis state $|j\rangle$), the amplitude at vertex $k$ after time $t$ is

$$\alpha_{jk}(t) = \langle k|U(t)|j\rangle = \sum_{m=0}^{n-1} e^{-i\lambda_m t}\, u_m(j)\, u_m(k).$$

The transfer probability is $p_{jk}(t) = |\alpha_{jk}(t)|^2$.

*Remark* 4.1. In the ISLS context, CTQW is not a physical quantum process. It is a *classical computation* using the mathematical formalism of quantum walks to measure how efficiently information propagates through the graph. The "quantum" label refers to the mathematical structure, not to hardware.

**Definition 4.7** (Propagation Speed)**.** The propagation speed from vertex $j$ to vertex $k$ is defined as

$$v_{jk} = \frac{d_G(j,k)}{t^*_{jk}},$$

where $d_G(j,k)$ is the shortest-path distance in $G$ and $t^*_{jk} = \arg\min_t\{t > 0 : p_{jk}(t) \geq \theta_{\text{prop}}\}$ is the first time the transfer probability exceeds threshold $\theta_{\text{prop}}$.

**Definition 4.8** (Mean Propagation Time)**.** The mean propagation time of the graph is

$$\bar{t}_{\text{prop}} = \frac{1}{|V|^2} \sum_{j,k\in V} t^*_{jk}.$$

**Definition 4.9** (CTQW Approximation)**.** For computational efficiency, the CTQW propagator is approximated using the spectral decomposition. Let $K \leq n$ be the number of retained eigenvalues (truncation rank). Then

$$U_K(t) = \sum_{m=0}^{K-1} e^{-i\lambda_m t}\, u_m u_m^\top.$$

For $K = \min(n, K_{\max})$ with configurable $K_{\max}$, this achieves $O(K^2 n)$ per time step instead of $O(n^3)$ for full matrix exponentiation.

### 4.3 Kuramoto Synchronization

**Definition 4.10** (Phase Assignment)**.** Each vertex $v_i$ carries a phase $\phi_i \in [0, 2\pi)$ derived from its 5D embedding:

$$\phi_i = \omega(\Phi(v_i)) \bmod 2\pi,$$

where $\omega(\cdot)$ extracts the frequency coordinate from the `FiveDState`.

**Definition 4.11** (Kuramoto Dynamics)**.** The Kuramoto model on the weighted graph evolves phases as

$$\dot{\phi}_i = \omega_i^{\text{nat}} + \frac{\kappa}{D_{ii}} \sum_{j:(i,j)\in E} \bar{W}_{ij} \sin(\phi_j - \phi_i),$$

where $\omega_i^{\text{nat}}$ is the natural frequency of vertex $i$ (derived from embedding) and $\kappa > 0$ is the global coupling strength.

**Definition 4.12** (Kuramoto Order Parameter)**.** The complex order parameter is

$$r\, e^{i\Psi} = \frac{1}{n} \sum_{j=1}^{n} e^{i\phi_j}.$$

The magnitude $r \in [0, 1]$ measures global phase coherence: $r = 1$ means perfect synchronization, $r = 0$ means complete incoherence. We define

$$r_k = |r\, e^{i\Psi}|$$

at macro-step $k$.

**Proposition 4.2** (Kuramoto Coherence and Mandorla)**.** *The Kuramoto order parameter $r_k$ provides a graph-intrinsic measure of phase coherence that is independent of the carrier geometry. When $r_k$ is high, the entities in the graph are phase-synchronized, which corresponds to a physically meaningful mandorla coherence. Therefore $r_k$ can serve as an alternative or complementary computation of the coherence metric $Q_k$.*

*Proof sketch.* The mandorla coupling $\kappa(t)$ in the carrier geometry measures interference between two helices. The Kuramoto $r_k$ measures interference among all entity phases. Both converge to the same qualitative judgment: high values indicate structural coherence, low values indicate incoherence. The carrier-geometric $Q_k$ and the graph-intrinsic $r_k$ are complementary views of the same phenomenon. □

### 4.4 Discrete Topology Logic (DTL)

**Definition 4.13** (DTL Predicates)**.** A DTL predicate is a Boolean function on graph structure. The minimal set of DTL predicates is:

(i) Connected($G$): the graph has exactly one connected component ($\Delta > 0$).
(ii) Bipartite($G$): the graph is bipartite ($\lambda_{n-1} = 2$ for $L_{\text{norm}}$, or all cycles have even length).
(iii) TreeLike($G$): $|E| = |V| - 1$ and $G$ is connected.
(iv) ClusterCount($G, k$): spectral clustering with $k$ clusters produces non-empty clusters (multiplicity of near-zero eigenvalues $\approx k$).
(v) Expander($G, c$): $\Delta \geq c$ (the graph is a $c$-spectral expander).

**Definition 4.14** (Topological Signature (Hardened))**.** The topological signature $\tau(C)$ of a crystal $C$ with condensed region $G^*$ is extended to include:

$$\tau(C) = \langle \beta_0, \beta_1, \beta_2, \Delta, \hat{h}, r_{\text{sync}}, \bar{t}_{\text{prop}}, \text{DTL}_{\text{vec}} \rangle$$

where:

- $\beta_0, \beta_1, \beta_2$: Betti numbers (existing),
- $\Delta$: spectral gap of the condensed region,
- $\hat{h}$: Cheeger constant estimate,
- $r_{\text{sync}}$: Kuramoto order parameter of the region,
- $\bar{t}_{\text{prop}}$: mean CTQW propagation time,
- $\text{DTL}_{\text{vec}}$: vector of DTL predicate results.

# 5 Integration with Existing Layers

## 5.1 Feeding Constraint Extraction (L2)

**Requirement 5.1** (Spectral Enrichment of Extraction)**.** Before `inverse_weave` runs, the topology crate **MUST** compute the spectral decomposition of the current graph and attach the spectral gap $\Delta$, Fiedler vector $u_1$, and Kuramoto $r_k$ to the extraction context. Constraint candidates **MAY** use these as additional features for coverage computation.

**Definition 5.1** (Spectral Constraint Templates)**.** Two new constraint template families are added:

(i) **SpectralGap**: $\Delta \geq \Delta_{\min}$. The graph maintains minimum connectivity.

(ii) **SyncCluster**: $r_k(\text{cluster}_i) \geq r_{\min}$. A cluster of entities maintains minimum phase synchronization.

## 5.2 Feeding Gate Evaluation (L3)

**Definition 5.2** (New Metrics M25–M27)**.** Three new metrics join the existing M1–M24:

- **M25 Spectral Gap**: $\Delta_k$ of the current graph. Alert threshold: $\Delta < 0.01$ (disconnection risk).
- **M26 Kuramoto Coherence**: $r_k$ of the full graph. Alert threshold: $r_k < 0.1$ (no synchronization).
- **M27 Mean Propagation Time**: $\bar{t}_{\text{prop},k}$. Alert threshold: $\bar{t}_{\text{prop}} > 100$ (signals propagate too slowly).

*Remark* 5.1. M25–M27 are *informational metrics*, not gate variables. They do not add new gates to the Kairos condition. They enrich the crystal topological signature and the metric dashboard. A future profile **MAY** promote them to gate variables if desired.

## 5.3 Feeding the Spiral Scheduler (C15)

**Requirement 5.2** (Topology-Informed Scheduling)**.** The spiral scheduler **MAY** use spectral gap and Kuramoto coherence as additional inputs to the scheduling function:

$$\sigma(d, F, S, \Delta, r_k, \theta_{\text{sched}}).$$

When $\Delta$ drops sharply (structural disconnection event) or $r_k$ drops sharply (desynchronization event), the scheduler **SHOULD** increase sub-step resolution.

## 5.4 Feeding Crystal Signatures

**Requirement 5.3** (Hardened Topological Signature)**.** Every crystal produced after C16 is active **MUST** include the hardened topological signature $\tau(C)$ as defined in definition 4.14. This extends the existing topology_signature field in the crystal schema.

# 6 Fixpoint Detection

Absorbed from `kernel-engine`'s `FixpointDetector`.

**Definition 6.1** (Jaccard Fixpoint Detection)**.** Let $V_k \subseteq V$ be the set of "active" vertex identifiers at macro-step $k$ (defined as vertices with at least one updated embedding in the current tick). The Jaccard distance between consecutive steps is

$$d_J(V_k, V_{k-1}) = 1 - \frac{|V_k \cap V_{k-1}|}{|V_k \cup V_{k-1}|}.$$

The system has reached a structural fixpoint when $d_J(V_k, V_{k-1}) \leq \epsilon_{\text{fix}}$ for $n_{\text{consec}}$ consecutive steps.

**Requirement 6.1** (Fixpoint Notification)**.** When fixpoint is detected, the topology crate **MUST**:
(a) emit a `FixpointReached` event to the audit log,
(b) notify the spiral scheduler (which **MAY** reduce resolution to $n_{\min}$),
(c) **NOT** halt the engine (fixpoints can be transient; the engine must continue processing new observations).

# 7 Deduplication

Absorbed from the `vacuum-pipeline` canonicalization stage.

**Definition 7.1** (Observation Deduplication)**.** Two observation records $o_1, o_2$ are duplicates if and only if $\text{Dig}(\text{Canon}(o_1)) = \text{Dig}(\text{Canon}(o_2))$. The deduplication filter is applied after canonicalization (L0) and before persistence (L1). Duplicate observations are logged but not ingested.

**Requirement 7.1** (Dedup Logging)**.** Every suppressed duplicate **MUST** be recorded in the audit log with: original digest, duplicate count, source identifier, timestamp. This preserves the evidence that duplicates were observed without polluting the graph.

# 8 Computational Complexity

| Operation | Complexity | Notes |
|---|---|---|
| Laplacian construction | $O(\|E\|)$ | Sparse matrix from edge list |
| Spectral decomposition | $O(K^2 n)$ | $K$-truncated eigendecomp via Lanczos iteration; $K \leq 50$ typical |
| CTQW propagation | $O(K^2 n \cdot T)$ | $T$ time steps, truncated spectral method |
| Kuramoto one step | $O(\|E\|)$ | Sparse neighbor sum |
| Kuramoto $N_{\text{steps}}$ | $O(N_{\text{steps}} \cdot \|E\|)$ | RK4 integration |
| Fixpoint detection | $O(\|V\|)$ | Jaccard on vertex sets |
| Deduplication | $O(1)$ amortized | Hash lookup in `BTreeSet` |

**Requirement 8.1** (Topology Budget)**.** A conformant implementation **MUST** declare a per-tick budget for topology computation. If the budget is exceeded (e.g., graph too large for full spectral decomposition), the implementation **MUST** fall back to:
(a) reduced truncation rank $K$,
(b) or sampling-based Kuramoto on a subgraph,
(c) or skipping CTQW propagation entirely (marking $\bar{t}_{\text{prop}}$ as `N/A` in the signature).
The fallback decision **MUST** be logged.

# 9 Rust API

```
// isls-topology/src/lib.rs

pub struct SpectralDecomposition {
    pub eigenvalues: Vec<f64>,           // sorted ascending
    pub eigenvectors: Vec<Vec<f64>>,     // column-major
    pub spectral_gap: f64,              // lambda_1
    pub cheeger_estimate: f64,          // sqrt(2 * lambda_1)
    pub fiedler_vector: Vec<f64>,       // u_1
    pub truncation_rank: usize,         // K
}

pub struct CtqwResult {
    pub transfer_probabilities: Vec<Vec<f64>>,  // p[j][k] at time t*
    pub propagation_speeds: Vec<Vec<f64>>,       // v[j][k]
    pub mean_propagation_time: f64,              // t_bar
}

pub struct KuramotoState {
    pub phases: Vec<f64>,               // phi_i for each vertex
    pub order_parameter: f64,           // r (magnitude)
    pub mean_phase: f64,                // Psi
    pub cluster_coherences: Vec<f64>,   // r per spectral cluster
}

pub struct TopologicalSignature {
    pub betti_numbers: Vec<u64>,        // beta_0, beta_1, beta_2
    pub spectral_gap: f64,
    pub cheeger_estimate: f64,
    pub kuramoto_coherence: f64,
    pub mean_propagation_time: f64,
    pub dtl_predicates: BTreeMap<String, bool>,
}

pub struct FixpointDetector {
    epsilon: f64,
    n_consecutive: usize,
    max_iterations: u64,
    // internal state...
}

pub enum FixpointStatus {
    Converged { iteration: u64 },
    NotYet { jaccard_distance: f64 },
    MaxIterations,
}

pub struct TopologyConfig {
```

```
    pub spectral_k_max: usize,          // max eigenvalues to compute
    pub ctqw_time_max: f64,             // max propagation time
    pub ctqw_threshold: f64,            // theta_prop
    pub ctqw_time_steps: usize,         // discretization steps
    pub kuramoto_coupling: f64,         // kappa
    pub kuramoto_steps: usize,          // integration steps per tick
    pub kuramoto_dt: f64,               // RK4 step size
    pub fixpoint_epsilon: f64,          // Jaccard convergence threshold
    pub fixpoint_consecutive: usize,    // required consecutive stable
    pub budget_ms: u64,                 // per-tick compute budget
}

// Core functions
pub fn compute_laplacian(graph: &PersistentGraph) -> SparseLaplacian;
pub fn spectral_decompose(
    laplacian: &SparseLaplacian,
    k_max: usize,
) -> SpectralDecomposition;

pub fn ctqw_propagate(
    spectral: &SpectralDecomposition,
    config: &TopologyConfig,
) -> CtqwResult;

pub fn kuramoto_step(
    state: &mut KuramotoState,
    graph: &PersistentGraph,
    config: &TopologyConfig,
);

pub fn kuramoto_order_parameter(phases: &[f64]) -> (f64, f64);

pub fn compute_topological_signature(
    graph: &PersistentGraph,
    config: &TopologyConfig,
) -> TopologicalSignature;

pub fn dtl_evaluate(
    graph: &PersistentGraph,
    spectral: &SpectralDecomposition,
) -> BTreeMap<String, bool>;

pub fn dedup_filter(
    observations: &[Observation],
    seen: &mut BTreeSet<Hash256>,
) -> (Vec<Observation>, usize);  // (unique, dup_count)
```

## 10 Acceptance Tests

| ID | Name | Procedure |
|---|---|---|
| AT-T1 | Laplacian correctness | Build 5-vertex graph; verify $L$ has zero row sums and correct eigenvalue count. |
| AT-T2 | Spectral gap connected | Build connected graph; verify $\Delta > 0$. Build disconnected graph; verify $\Delta = 0$. |
| AT-T3 | CTQW self-return | Initialize CTQW at vertex $j$ at $t = 0$; verify $p_{jj}(0) = 1$. |
| AT-T4 | CTQW unitarity | Verify $\sum_k p_{jk}(t) = 1$ for all $j$ and sampled $t$ values (within floating-point tolerance). |
| AT-T5 | Kuramoto convergence | Initialize identical phases; verify $r = 1$. Initialize random phases with strong coupling; verify $r \to 1$ over sufficient steps. |
| AT-T6 | Kuramoto incoherence | Initialize uniformly distributed phases with zero coupling ($\kappa = 0$); verify $r \approx 0$. |
| AT-T7 | DTL connected predicate | Build connected and disconnected graphs; verify DTL predicate matches spectral gap. |
| AT-T8 | Fixpoint detection | Feed identical vertex sets for $n_{\text{consec}}$ iterations; verify `Converged`. Feed changing sets; verify `NotYet`. |
| AT-T9 | Deduplication | Ingest 100 observations with 20 duplicates; verify 80 pass through, 20 logged as duplicates. |
| AT-T10 | Topological signature determinism | Compute signature twice on same graph; verify bitwise identical. |
| AT-T11 | Budget fallback | Set budget to 1ms on a large graph; verify fallback is used and logged. |
| AT-T12 | Crystal signature enrichment | Run S-Basic with topology enabled; verify crystals contain spectral gap, Kuramoto $r$, and mean propagation time in $\tau(C)$. |

# Part III
# Crate C17: `isls-store`

## 11 Purpose

The store crate upgrades ISLS from filesystem-based persistence (`~/.isls/` with JSONL files) to a structured SQLite database. This enables: project management, run indexing, crystal querying, settings persistence, and maintenance operations. It is the foundation for any future GUI, REST API, or multi-user deployment.

## 12 Data Model

**Definition 12.1** (Store Entities)**.** The store manages the following entity types:

- **Project**: a named workspace containing runs, crystals, and configuration.
- **Run**: a single execution (discover or execute mode) with associated manifest.
- **Crystal**: a committed semantic crystal with all fields from the crystal schema.

- **Trace**: per-tick trace entries for a run.
- **Manifest**: execution manifest for a run.
- **Capsule**: sealed capsule objects.
- **Registry Snapshot**: point-in-time copy of all registries used in a run.
- **Metric**: time-series metric snapshots.
- **Alert**: emitted alert events.
- **Setting**: key-value configuration.

## 13 Schema

```
-- isls-store schema v1

CREATE TABLE projects (
    id          TEXT PRIMARY KEY,   -- content-addressed
    name        TEXT NOT NULL,
    created_at  TEXT NOT NULL,      -- ISO 8601
    description TEXT DEFAULT ''
);

CREATE TABLE runs (
    id            TEXT PRIMARY KEY,   -- manifest.run_id
    project_id    TEXT NOT NULL REFERENCES projects(id),
    mode          TEXT NOT NULL,      -- 'discover' | 'execute'
    rd_digest     TEXT NOT NULL,
    started_at    TEXT NOT NULL,
    finished_at   TEXT,
    ticks         INTEGER NOT NULL,
    crystal_count INTEGER DEFAULT 0,
    status        TEXT DEFAULT 'running' -- 'running'|'completed'|'failed'
);

CREATE TABLE crystals (
    crystal_id         TEXT PRIMARY KEY,
    run_id             TEXT NOT NULL REFERENCES runs(id),
    stability_score    REAL NOT NULL,
    free_energy        REAL NOT NULL,
    created_at_tick    INTEGER NOT NULL,
    carrier_instance   INTEGER NOT NULL,
    constraint_count   INTEGER NOT NULL,
    region_size        INTEGER NOT NULL,
    topology_signature TEXT NOT NULL,    -- JSON
    validation_status  TEXT DEFAULT 'pending', -- 'pending'|'passed'|'failed'
    data               TEXT NOT NULL     -- full crystal JSON
);

CREATE TABLE traces (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    run_id          TEXT NOT NULL REFERENCES runs(id),
    tick            INTEGER NOT NULL,
```

```
    input_digest    TEXT NOT NULL,
    state_digest    TEXT NOT NULL,
    crystal_id      TEXT,               -- NULL if no crystal this tick
    gate_snapshot   TEXT NOT NULL,       -- JSON
    metrics_digest  TEXT NOT NULL,
    UNIQUE(run_id, tick)
);

CREATE TABLE manifests (
    run_id           TEXT PRIMARY KEY REFERENCES runs(id),
    manifest_json    TEXT NOT NULL,      -- full manifest JSON
    verification     TEXT DEFAULT 'pending'
);

CREATE TABLE capsules (
    id              TEXT PRIMARY KEY,    -- content-addressed
    run_id          TEXT REFERENCES runs(id),
    policy_json     TEXT NOT NULL,
    created_at      TEXT NOT NULL,
    opened_count    INTEGER DEFAULT 0,
    max_uses        INTEGER,
    expires_at      TEXT
);

CREATE TABLE registries (
    id              TEXT PRIMARY KEY,
    run_id          TEXT NOT NULL REFERENCES runs(id),
    kind            TEXT NOT NULL,       -- 'operator'|'profile'|...
    digest          TEXT NOT NULL,
    data            TEXT NOT NULL        -- full registry JSON
);

CREATE TABLE metrics (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    run_id          TEXT NOT NULL REFERENCES runs(id),
    tick            INTEGER NOT NULL,
    snapshot_json   TEXT NOT NULL,       -- full MetricSnapshot JSON
    UNIQUE(run_id, tick)
);

CREATE TABLE alerts (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    run_id          TEXT NOT NULL REFERENCES runs(id),
    tick            INTEGER NOT NULL,
    metric_id       TEXT NOT NULL,
    level           TEXT NOT NULL,       -- 'green'|'yellow'|'red'
    message         TEXT NOT NULL
);
```

```
CREATE TABLE settings (
    key             TEXT PRIMARY KEY,
    value           TEXT NOT NULL,
    updated_at      TEXT NOT NULL
);
```

## 14 Invariant Preservation

**Invariant 14.1** (Store Append-Only Crystals)**.** The crystals table **MUST NOT** support `UPDATE` or `DELETE` on the `crystal_id`, `data`, `stability_score`, `free_energy`, or `topology_signature` columns. Only `validation_status` may be updated (from `pending` to `passed` or `failed`). This preserves Invariant I10 (Commit Immutability).

**Invariant 14.2** (Store Deterministic Queries)**.** All queries used in replay-relevant paths **MUST** include explicit `ORDER BY` clauses on deterministic columns (`crystal_id`, `tick`, or `id`). No query may depend on insertion order or SQLite internal rowid ordering for correctness.

**Requirement 14.1** (Migration Framework)**.** The store **MUST** include a migration framework that:

(a) tracks applied migrations by version number,
(b) applies migrations in order at startup,
(c) is idempotent (re-running a migration on an already-migrated database is a no-op),
(d) preserves all existing data during schema evolution.

## 15 Rust API

```
// isls-store/src/lib.rs

pub struct IslandStore {
    conn: Mutex<rusqlite::Connection>,
}

impl IslandStore {
    pub fn open(path: &Path) -> Result<Self>;
    pub fn open_memory() -> Result<Self>;  // for tests
    pub fn migrate(&self) -> Result<()>;

    // Projects
    pub fn create_project(&self, name: &str, desc: &str) -> Result<String>;
    pub fn list_projects(&self) -> Result<Vec<Project>>;
    pub fn get_project(&self, id: &str) -> Result<Project>;

    // Runs
    pub fn create_run(&self, project_id: &str, mode: &str,
                      rd_digest: &str, ticks: u64) -> Result<String>;
    pub fn finish_run(&self, run_id: &str, crystal_count: u64) -> Result<()>;
    pub fn get_run(&self, run_id: &str) -> Result<Run>;
    pub fn list_runs(&self, project_id: &str) -> Result<Vec<Run>>;

    // Crystals (append-only)
```

```
    pub fn insert_crystal(&self, crystal: &CrystalRow) -> Result<()>;
    pub fn get_crystal(&self, crystal_id: &str) -> Result<CrystalRow>;
    pub fn list_crystals(&self, run_id: &str) -> Result<Vec<CrystalRow>>;
    pub fn update_validation(&self, crystal_id: &str,
                             status: &str) -> Result<()>;

    // Traces
    pub fn insert_trace(&self, trace: &TraceRow) -> Result<()>;
    pub fn get_traces(&self, run_id: &str) -> Result<Vec<TraceRow>>;

    // Manifests
    pub fn insert_manifest(&self, run_id: &str,
                           json: &str) -> Result<()>;
    pub fn get_manifest(&self, run_id: &str) -> Result<String>;

    // Capsules
    pub fn insert_capsule(&self, capsule: &CapsuleRow) -> Result<()>;
    pub fn increment_opened(&self, capsule_id: &str) -> Result<()>;

    // Metrics
    pub fn insert_metric(&self, run_id: &str, tick: u64,
                         snapshot: &str) -> Result<()>;
    pub fn get_metrics(&self, run_id: &str) -> Result<Vec<MetricRow>>;
    pub fn get_latest_metric(&self, run_id: &str) -> Result<MetricRow>;

    // Alerts
    pub fn insert_alert(&self, alert: &AlertRow) -> Result<()>;
    pub fn get_alerts(&self, run_id: &str) -> Result<Vec<AlertRow>>;

    // Settings
    pub fn set_setting(&self, key: &str, value: &str) -> Result<()>;
    pub fn get_setting(&self, key: &str) -> Result<Option<String>>;

    // Maintenance
    pub fn vacuum(&self) -> Result<()>;
    pub fn integrity_check(&self) -> Result<bool>;
    pub fn export_run_zip(&self, run_id: &str,
                          path: &Path) -> Result<()>;
}
```

## 16 CLI Integration

```
# Initialize store (creates ~/.isls/isls.db)
isls init --store sqlite

# List projects
isls project list

# Create project
```

```
isls project create --name "market-analysis"

# Run within a project
isls run --mode shadow --ticks 100 --project market-analysis

# Query crystals
isls crystal list --run <run-id>
isls crystal show <crystal-id>

# Export a run as ZIP (manifest + crystals + traces + metrics)
isls export --run <run-id> --output run-export.zip

# Maintenance
isls store vacuum
isls store check
```

## 17 Acceptance Tests

| ID | Name | Procedure |
|---|---|---|
| AT-D1 | Create and query project | Create project; list; verify present. |
| AT-D2 | Crystal append-only | Insert crystal; attempt UPDATE on data column; verify rejection. |
| AT-D3 | Run lifecycle | Create run; insert crystals; finish run; verify crystal_count matches. |
| AT-D4 | Manifest round-trip | Insert manifest JSON; retrieve; verify identical. |
| AT-D5 | Trace ordering | Insert traces out of order; query; verify returned in tick order. |
| AT-D6 | Migration idempotent | Run migrate twice; verify no error and same schema. |
| AT-D7 | Integrity check | Populate database; run integrity_check; verify true. |
| AT-D8 | Export ZIP | Run S-Basic into store; export ZIP; verify contains manifest.json, crystals.jsonl, traces.jsonl. |

# Part IV
# Dependency Graph and Build Order

## 18 Updated Dependency Graph

```
isls-types        <-- (all crates)
    ^
isls-registry     <-- (all crates that use operators)
    ^
```

```
isls-observe  <-- isls-persist <-- isls-topology (C16)
                        ^                ^
                        |          isls-extract (uses spectral features)
                        |                ^
                isls-consensus <-- isls-carrier
                        ^
                isls-archive <-- isls-morph
                        ^
                isls-manifest
                        ^
                isls-capsule
                        ^
                isls-scheduler (uses fixpoint from C16)
                        ^
                isls-engine (+ execute + fixpoint integration)
                        ^
                isls-store (C17: SQLite, depends on types + archive)
                        ^
                isls-harness (extended with topology tests)
                        ^
                isls-cli (extended with store commands)
```

## 19 Build Order

1. `isls-topology` (C16): depends on `isls-types`, `isls-persist`. New external dependency: `nalgebra` (already in workspace for linear algebra).
2. `isls-store` (C17): depends on `isls-types`, `isls-archive`. New external dependency: `rusqlite` with `bundled` feature.
3. Engine integration: wire topology into `macro_step` (spectral enrichment before extraction, Kuramoto before gate evaluation).
4. CLI integration: add store commands, topology metrics in reports.
5. Harness integration: add AT-T1 through AT-T12 and AT-D1 through AT-D8.

## 20 External Dependencies

| Crate | Version | Purpose |
|---|---|---|
| `nalgebra` | 0.33 | Eigendecomposition, matrix exponential |
| `rusqlite` | 0.32 | SQLite database (bundled) |

# Part V
# Implementation Handoff

## 21 Test Count

| Phase | New Tests | Cumulative |
|---|---|---|
| Core (C1–C9) | 130 | 130 |
| Extension Phase 1 (C10–C15) | 31 | 161 |
| Extension Phase 2 (C16–C17) | 20 | $\geq$ 181 |

## 22 Verification Sequence

```
# Build
cargo build --workspace --release 2>&1 | grep -c warning  -> 0

# Test
cargo test --workspace -- --test-threads=1  -> >= 181 passed

# S-Basic with topology enrichment
rm -rf ~/.isls
cargo run --bin isls --release -- init --store sqlite
cargo run --bin isls --release -- project create --name test
cargo run --bin isls --release -- ingest --adapter synthetic --scenario S-Basic
cargo run --bin isls --release -- run --mode shadow --ticks 100 --project test
cargo run --bin isls --release -- validate formal

# Verify topology in crystal signatures
# Crystals must contain spectral_gap, kuramoto_coherence,
# mean_propagation_time in topology_signature

# Export
cargo run --bin isls --release -- export --run latest --output test.zip
# Verify ZIP contains manifest.json, crystals.jsonl, traces.jsonl
```

## 23 Completion Criteria

Phase 2 is complete when:

1. `isls-topology` compiles, all AT-T tests pass.
2. `isls-store` compiles, all AT-D tests pass.
3. Crystals from S-Basic contain hardened topological signatures with $\Delta$, $r_k$, $\bar{t}_{\text{prop}}$.
4. M25–M27 appear in metric snapshots and HTML reports.
5. Store CLI commands work (init, project, crystal list, export).
6. Total tests $\geq$ 181, zero warnings.
7. Full scenario script runs end-to-end with store backend.

---

Generated for ISLS Extension Phase 2 v1.0.0.
Deterministic, append-only, replay-verified.